

adapting a real app for

IPHONE DUO

EPISODE 1: TOOLBARS

Every code snippet from the video, before and after, in order.

by Craig Clayton

HOW TO READ THIS

BEFORE the code as it was in Countdown2Binge.

AFTER the code after the change.

NEW code that did not exist before.

ERRORS compiler output, not code to copy.

Snippets that pair up say so, for example "Pairs with 02."

axisBehavior needs iOS 27.1 and visibilityPriority needs iOS 27.0. Snippet 17 shows how to guard them for iOS 26.

Compiled against the Xcode 27.1 RC SDK, with stand-in types for the app's own models.

01 The Toolbar That Already Works

NO CHANGE

FollowedShowDetail.swift. The app's real toolbar. It needed no change.

```
.toolbar {
    // Alerts bell: premium only
    if PremiumManager.shared.isPremium {
        ToolbarItem(placement: .topBarTrailing) {
            Button {
                showAlerts = true
            } label: {
                Image(systemName: series.notificationsActive ? "bell.fill" : "bell.slash.fill")
                    .resizable()
                    .scaledToFit()
                    .frame(width: 20, height: 20)
                    .foregroundColor(series.notificationsActive ? .c2bTealBright : .c2bMuted)
            }
        }
    }
}

ToolbarItem(placement: .topBarTrailing) {
    Menu {
        // Watch Now
        if let network = show.networks.first,
           let service = StreamingService.from(networkId: network.id, networkName: network.name) {
            Button {
                openStreamingApp(service: service)
            } label: {
                Label(String(localized: "watch_on \(service.displayName)", systemImage: "play.tv")
            }
        }

        // Share
        Button {
            showShareSheet = true
        } label: {
            Label(String(localized: "button_share"), systemImage: "square.and.arrow.up")
        }

        // Archive / Unarchive
        Button {
            toggleArchive()
        } label: {
            Label(
                isArchived ? String(localized: "button_unarchive") : String(localized: "button_archive"),
                systemImage: isArchived ? "arrow.eturn.backward" : "archivebox"
            )
        }
    } label: {
        Image(systemName: "ellipsis")
            .resizable()
            .scaledToFit()
            .frame(width: 24, height: 24)
    }
}
.sharedBackgroundVisibility(.hidden)
}
```

02 My Library Header

BEFORE

MyLibraryScreen.swift. The title, and the Done button or the toggle, drawn in the scroll content.

```
// My Library's header, before
private var header: some View {
    HStack {
        Text("library_title")
        Spacer()
        if viewModel.isEditMode {
            Button(action: { viewModel.exitEditMode() }) {
                Text("library_done")
            }
        } else {
            MyLibraryViewModeToggle(mode: $viewModel)
        }
    }
}
```

03 My List Header

BEFORE

MyListLandscapeView.swift. The title and the refresh button, drawn in the scroll content.

```
// My List's header, before
private var header: some View {
    VStack(alignment: .leading, spacing: 0) {
        HStack(alignment: .top) {
            Text(String(localized: "mylist_ls_title").uppercased())
            Spacer()
            Button(action: { cloudSettings.hasSeenMyListOnboarding = false }) {
                Image(systemName: "arrow.counterclockwise")
            }
        }
        readyStatLine
        .padding(.top, 8)
    }
}
```

04 My Library: The Toggle in a Toolbar

AFTER

MyLibraryScreen.swift. Pairs with 02.

```
.toolbar {
    ToolbarItem(placement: .topBarTrailing) {
        if viewModel.isEditMode {
            Button(action: { viewModel.exitEditMode() }) { Text("library_done") }
        } else {
            MyLibraryViewModelToggle(mode: $viewModel)
        }
    }
}
```

05 My Library Header, Trimmed

AFTER

MyLibraryScreen.swift. Only the title is left. Pairs with 02.

```
private var header: some View {
    HStack {
        Text("library_title")
            .font(.custom(.oswald.bold, size: 27))
        Spacer()
    }
}
```

06 Asking for the Side: axisBehavior

AFTER

MyLibraryScreen.swift. Needs iOS 27.1.

NOTE

The toggle moves to the side of the screen, but it stays a wide pill. A custom view does not reshape itself, so this is a step on the way, not the finished fix.

```
.toolbar {
    if viewModel.isEditMode {
        ToolbarItem(placement: .topBarTrailing) {
            Button(action: { viewModel.exitEditMode() }) { Text("library_done") }
        }
    }

    if !viewModel.isEditMode {
        ToolbarItem(placement: .topBarTrailing) {
            MyLibraryViewModeToggle(mode: $viewModel)
        }
        .axisBehavior(.verticalPreferred)
    }
}
```

07 My Library: One Icon AFTER

MyLibraryScreen.swift. The toggle becomes a single icon button, with no axis modifier.

NOTE

An image item can sit in the vertical bar on its own. Done is still text here, and text needs the horizontal bar, which pushes the screen down in edit mode.

```
.toolbar {
    if viewModel.isEditMode {
        ToolbarItem(placement: .topBarTrailing) {
            Button(action: { viewModel.exitEditMode() }) {
                Text("library_done")
            }
        }
    }
    if !viewModel.isEditMode {
        ToolbarItem(placement: .topBarTrailing) {
            Button {
                viewModel = viewModel == .grid ? .list : .grid
            } label: {
                Image(systemName: viewModel == .grid ? "line.3.horizontal" : "square.grid.2x2")
            }
            .accessibilityLabel(viewModel == .grid ? "Switch to list view" : "Switch to grid view")
        }
    }
}
```

08 Done as a Checkmark AFTER

MyLibraryScreen.swift. Replaces the Done button inside the toolbar from 07.

NOTE

The checkmark is an image, so it stays in the side strip. Entering edit mode no longer adds a horizontal bar. VoiceOver still says Done.

```
if viewModel.isEditMode {
    ToolbarItem(placement: .topBarTrailing) {
        Button(action: { viewModel.exitEditMode() }) {
            Image(systemName: "checkmark")
        }
        .accessibilityLabel(Text("library_done"))
    }
}
```

09 My List: The Toolbar

AFTER

MyListLandscapeView.swift. Pairs with 03.

```
.toolbar {
    ToolbarItem(placement: .topBarTrailing) {
        Button(action: { cloudSettings.hasSeenMyListOnboarding = false }) {
            Image(systemName: "arrow.counterclockwise")
        }
        .accessibilityLabel(Text("Redo Onboarding"))
    }
}
```

10 My List Header, Trimmed

AFTER

MyListLandscapeView.swift. Pairs with 03.

```
private var header: some View {
    VStack(alignment: .leading, spacing: 0) {
        Text(String(localized: "mylist_ls_title").uppercased())
            .font(.custom(.oswald.bold, size: 27))
        readyStatLine
            .padding(.top, 8)
    }
}
```

11 TimelineHeader

BEFORE

TimelineHeader.swift. The bell and info buttons, drawn in the header.

```
// TimelineHeader, before
HStack(spacing: 12) {
    // avatar, salutation, name
    Spacer()

    if PremiumManager.shared.isPremium {
        Button(action: onBellTap) {
            Image(systemName: "bell")
                .frame(width: 42, height: 42)
                .background(Color.white.opacity(0.04))
                .cornerRadius(21)
                .overlay(Circle().stroke(Color.white.opacity(0.1), lineWidth: 1))
        }
    }

    if let onInfoTap {
        Button(action: onInfoTap) {
            Image(systemName: "info")
                .frame(width: 42, height: 42)
                .background(Color.white.opacity(0.04))
                .cornerRadius(21)
                .overlay(Circle().stroke(Color.white.opacity(0.1), lineWidth: 1))
        }
    }
}
```

12 TimelineScreen

BEFORE

TimelineScreen.swift. How the screen called the header.

```
// TimelineScreen, before
TimelineHeader(
    userName: profile.displayName,
    onBellTap: {
        if PremiumManager.shared.isPremium {
            showNotificationSettings = true
        }
    },
    onInfoTap: onInfoTap,
    lastUpdated: nil
)
```

13 TimelineScreen AFTER

TimelineScreen.swift. The toolbar goes on the scroll view, inside the NavigationStack. Pairs with 12.

```
// TimelineScreen, after
NavigationStack(path: $navigationPath) {
    ScrollView {
        // header, stats, hero cards...
    }
    .toolbar {
        if PremiumManager.shared.isPremium {
            ToolbarItem(placement: .topBarTrailing) {
                Button {
                    showNotificationSettings = true
                } label: {
                    Image(systemName: "bell")
                }
            }
        }

        if let onInfoTap {
            ToolbarItem(placement: .topBarTrailing) {
                Button(action: onInfoTap) {
                    Image(systemName: "info")
                }
            }
        }
    }
}
```

14 TimelineHeader AFTER

TimelineHeader.swift. The two buttons and their callbacks are gone. Pairs with 11.

```
// TimelineHeader, after
HStack(spacing: 12) {
    // avatar, salutation, name
    Spacer()
}
```

15 Priority for the Bell

AFTER

FollowedShowDetail.swift. Needs iOS 27.0.

NOTE

In the app this is applied through `keepVisibleIfAvailable()` from snippet 17, so the app still builds for iOS 26.

```
.toolbar {
    if PremiumManager.shared.isPremium {
        ToolbarItem(placement: .topBarTrailing) {
            Button {
                showAlerts = true
            } label: {
                Image(systemName: series.notificationsActive ? "bell.fill" : "bell.slash.fill")
                // same image modifiers as before
            }
        }
        .visibilityPriority(.high)
    }

    // the ellipsis Menu item is unchanged
}
```

16 The Two Compiler Errors

ERRORS

What you get at a deployment target of iOS 26.

```
'visibilityPriority' is only available in iOS 27.0 or newer
'axisBehavior' is only available in iOS 27.1 or newer
```

17 The Availability Helpers

NEW

Utilities/Extension+ToolbarContent.swift

NOTE

Check availability once, here, and call the helper everywhere else. On iOS 27 and later the modifier applies. On iOS 26 the item is left as it was.

```
extension ToolbarContent {
    /// Keeps this toolbar item visible longer when space runs short, on iOS 27 and later.
    @ToolbarContentBuilder
    func keepVisibleIfAvailable() -> some ToolbarContent {
        if #available(iOS 27.0, *) {
            self.visibilityPriority(.high)
        } else {
            self
        }
    }

    /// Lets this toolbar item sit in the vertical bar, on iOS 27.1 and later.
    @ToolbarContentBuilder
    func preferVerticalBarIfAvailable() -> some ToolbarContent {
        if #available(iOS 27.1, *) {
            self.axisBehavior(.verticalPreferred)
        } else {
            self
        }
    }
}
```

18 Call Site: The Bell

AFTER

FollowedShowDetail.swift. Uses the helper from 17.

```
// the bell on the show detail screen
.toolbar {
    if PremiumManager.shared.isPremium {
        ToolbarItem(placement: .topBarTrailing) {
            Button { showAlerts = true } label: { Image(systemName: "bell") }
        }
        .keepVisibleIfAvailable()
    }
}
```

19 Call Site: The Toggle

AFTER

MyLibraryScreen.swift. The version that used `axisBehavior`, before it became an icon. Uses the helper from 17.

```
// the second version of My Library, before I swapped the toggle for an icon
.toolbar {
    if !viewModel.isEditMode {
        ToolbarItem(placement: .topBarTrailing) {
            MyLibraryViewModelToggle(mode: $viewModel)
        }
        .preferVerticalBarIfAvailable()
    }
}
```